

Java Sorting Notes (Explanation, Theory, and Code)

1. What is Sorting?

Definition: Sorting is the process of arranging data in a particular order (ascending or descending).

Use case:

- Improves search efficiency (e.g., Binary Search needs sorted data).
 - Arranges records systematically.
-

2. Bubble Sort

Theory: Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. Largest elements “bubble up” to the end.

Algorithm:

- Compare adjacent pairs.
- Swap if needed.
- Repeat the process for (n-1) passes.

Code:

```
public class BubbleSort {
    public static void main(String[] args) {
        int[] arr = {5, 1, 4, 2, 8};
        bubbleSort(arr);
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }

    static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    // Swap
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

```
}  
}
```

3. Selection Sort

Theory: Selection Sort finds the minimum element from the unsorted part and places it at the beginning.

Algorithm:

- Find the minimum in unsorted part.
- Swap with the first unsorted element.
- Repeat for all positions.

Code:

```
public class SelectionSort {  
    public static void main(String[] args) {  
        int[] arr = {64, 25, 12, 22, 11};  
        selectionSort(arr);  
        for (int num : arr) {  
            System.out.print(num + " ");  
        }  
    }  
  
    static void selectionSort(int[] arr) {  
        int n = arr.length;  
        for (int i = 0; i < n - 1; i++) {  
            int minIndex = i;  
            for (int j = i + 1; j < n; j++) {  
                if (arr[j] < arr[minIndex]) {  
                    minIndex = j;  
                }  
            }  
            // Swap  
            int temp = arr[minIndex];  
            arr[minIndex] = arr[i];  
            arr[i] = temp;  
        }  
    }  
}
```

4. Insertion Sort

Theory: Insertion Sort builds the sorted array one item at a time by inserting current elements at their correct position.

Algorithm:

- Start from index 1.
- Compare with previous elements.
- Insert at the correct position.

Code:

```
public class InsertionSort {
    public static void main(String[] args) {
        int[] arr = {4, 3, 2, 10, 12, 1, 5, 6};
        insertionSort(arr);
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }

    static void insertionSort(int[] arr) {
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }
    }
}
```

5. Inbuilt Sort (Arrays.sort)

Ascending Sort:

```
import java.util.Arrays;

public class InbuiltSortAsc {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 1, 3};
        Arrays.sort(arr);
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }
}
```

Explanation: Uses Dual-Pivot QuickSort for primitives, TimSort for objects.

Descending Sort:

```
import java.util.*;

public class InbuiltSortDesc {
    public static void main(String[] args) {
        Integer[] arr = {5, 2, 8, 1, 3};
        Arrays.sort(arr, Collections.reverseOrder());
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }
}
```

Explanation: Needs Integer[] (not int[]) because Collections.reverseOrder() works on objects only.

6. Counting Sort

Theory: Counting Sort is a non-comparison-based sorting algorithm suitable for sorting small range integers.

Algorithm:

- Count frequency of each element.
- Update count array.
- Place elements accordingly.

Code:

```
public class CountingSort {
    public static void main(String[] args) {
        int[] arr = {4, 2, 2, 8, 3, 3, 1};
        countingSort(arr);
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }

    static void countingSort(int[] arr) {
        int max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
    }
}
```

```
    }  
}  
  
int[] count = new int[max + 1];  
for (int num : arr) {  
    count[num]++;  
}  
  
int index = 0;  
for (int i = 0; i < count.length; i++) {  
    while (count[i] > 0) {  
        arr[index++] = i;  
        count[i]--;  
    }  
}  
}
```
